Between you and the people around you, how many different languages do you speak in total?

clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

Threading

Updates

1. MP4 Allocator due tonight

1. MP5 Wallet out today (start after Thursday's lecture)

1. Exam 2 sign up open

1. HW 6 due Wednesday

MP 3.1
MP 3.2
MP 4

Threading

Today's LGs - Expand your mental model of a program to include multiple concurrent threads.

Be able to explain the difference between a thread and a process

Be able to reason about a multithreaded program to figure out

- Possible outputs
- If there is a race condition

Agenda

1. Big Picture Perspective
2. Processes
3. Threading
    a. Race Conditions
4. Synchronization in Programming

Big Picture Perspective

Limitations of Reality

resources are limited
things do not happen instantly
some tasks have dependencies

System - a collection of components that work together

↑ complexity of a system ~~[illegible]~~ the more it cares about

Non-Computer System Example

My kitchen

time

order

1) boil water takes 5 mins

2) I have to put eggs, flour in a pan before I bake it

3) I have only 1 pan

A Computer as a System

- many parts - memory, disk, cpu...

limited resources - cpu can only run 1 instruction at a time

things take time - moving bytes from disk to memory

things have dependencies - can't look up PA before loading the page table

---

Safety + efficiency

- virtual memory, processes, context switching

From Last Time

Process - running instance of a program

- Page table (virtual memory space)
- register values
- program counter

Kernel - part of the OS

that has special access

ex) handles page tables, context switching

What prints?

main.c

```c
#include <stdio.h>
#include <unistd.h>

unsigned int calc(int n){
    unsigned int ret = 1;
    for(int i = n; i > 0; i--){
        ret = i * ret;
    }
    return ret;
}

int main(int argc, char *argv[]) {
    sleep(3); // Pauses for 3 seconds
    int intValue = argv[1][0] - '0';
    printf("%i", calc(intValue));
}
```

```
gcc main.c
./a.out 4
```

clicker.cs.illinois.edu

Q2

~Code~
340



main.c

```c
#include <stdio.h>
#include <unistd.h>

unsigned int calc(int n){
    unsigned int ret = 1;
    for(int i = n; i > 0; i--){
        ret = i * ret;
    }
    return ret;
}

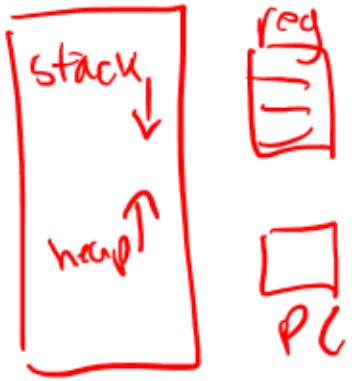
int main(int argc, char *argv[]) {
    sleep(3); // Pauses for 3 seconds
    int intValue = argv[1][0] - '0';
    printf("%i", calc(intValue));
}
```

```
gcc main.c
```

```
./a.out 3
```



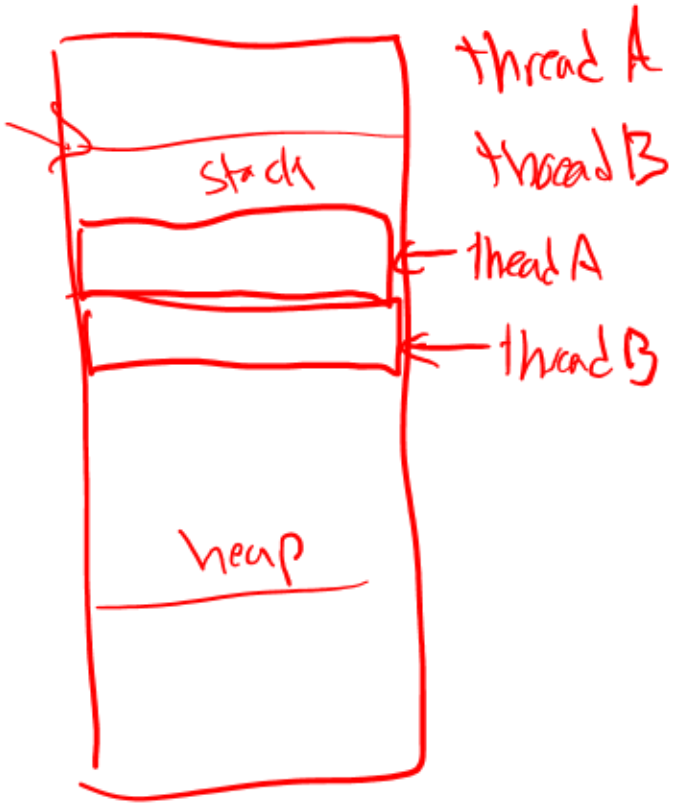

```
./a.out 4
```



Thread - an ~~ista~~ instance of execution within a process

each process has 1-many threads process A





CPU Scheduling

Kernel decides if it should context switch when:

- Timer interrupt
- Exception
- Sys call

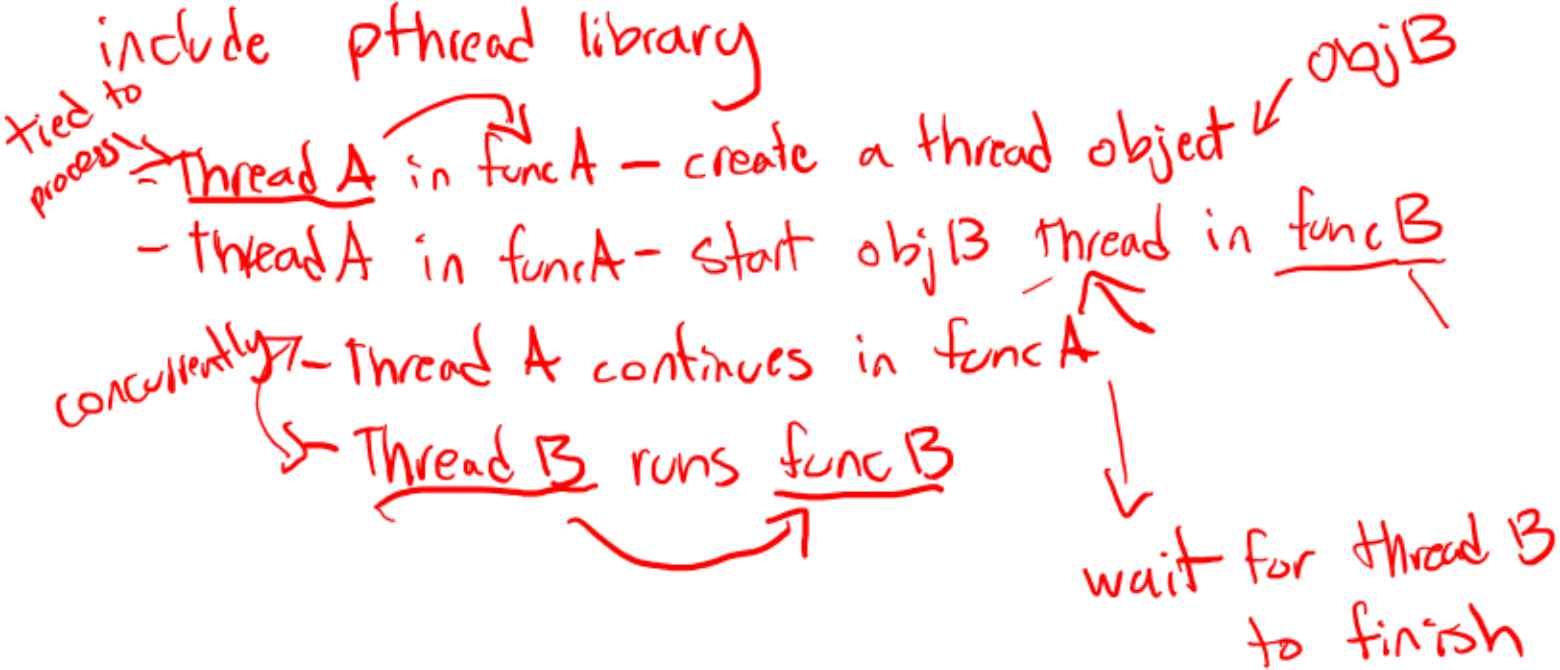
Context switching is on a thread level

Process A
- Thread 1A
- Thread 2A

Process B
- Thread 1B

1A is in the cpu
2A is switched in ← PC and reg switch
1B is switched in ← PC, reg, page table

Threads in C - Example format



Thread Example

```
gcc main.c
./a.out
```

```c
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

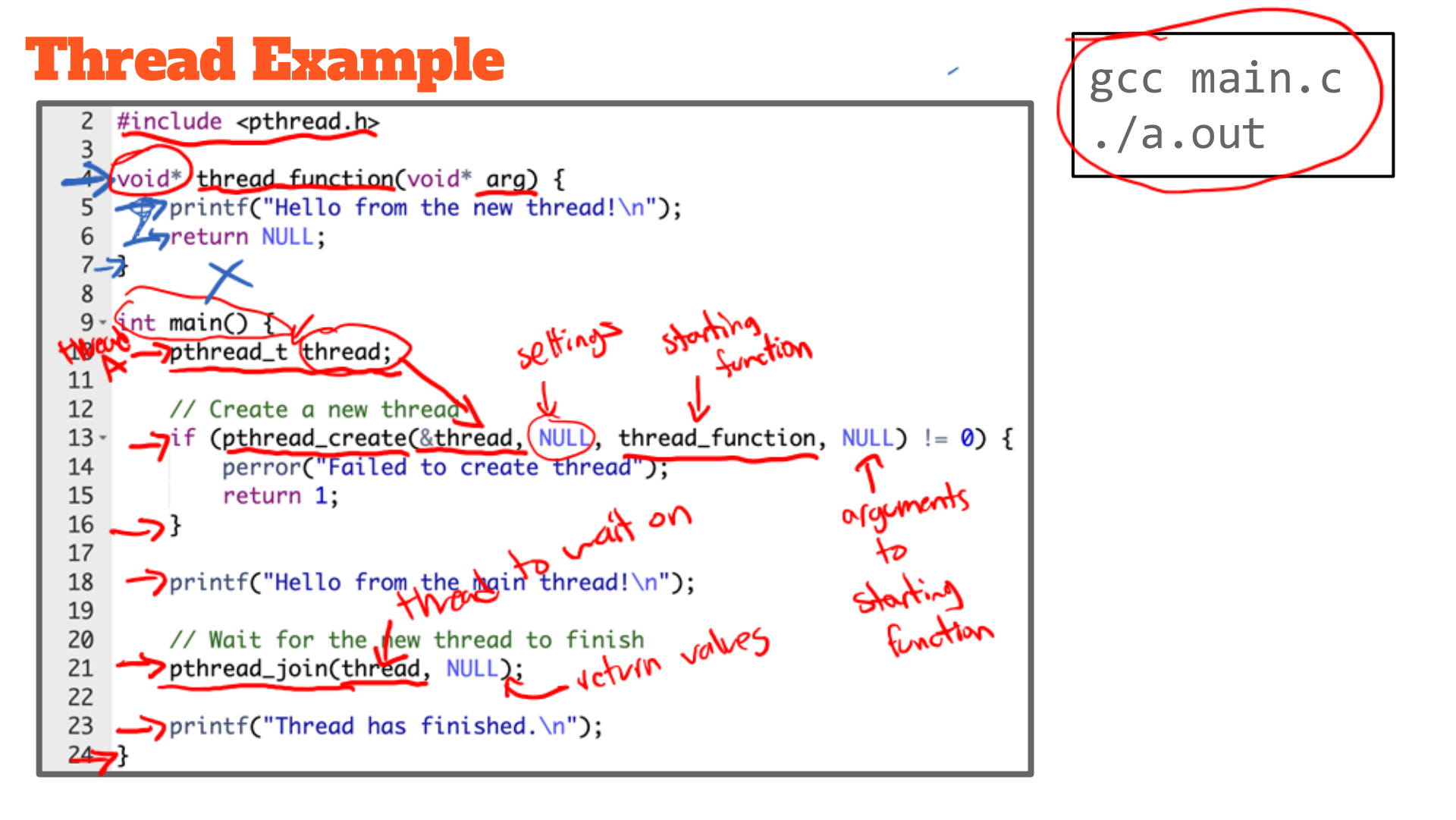
int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

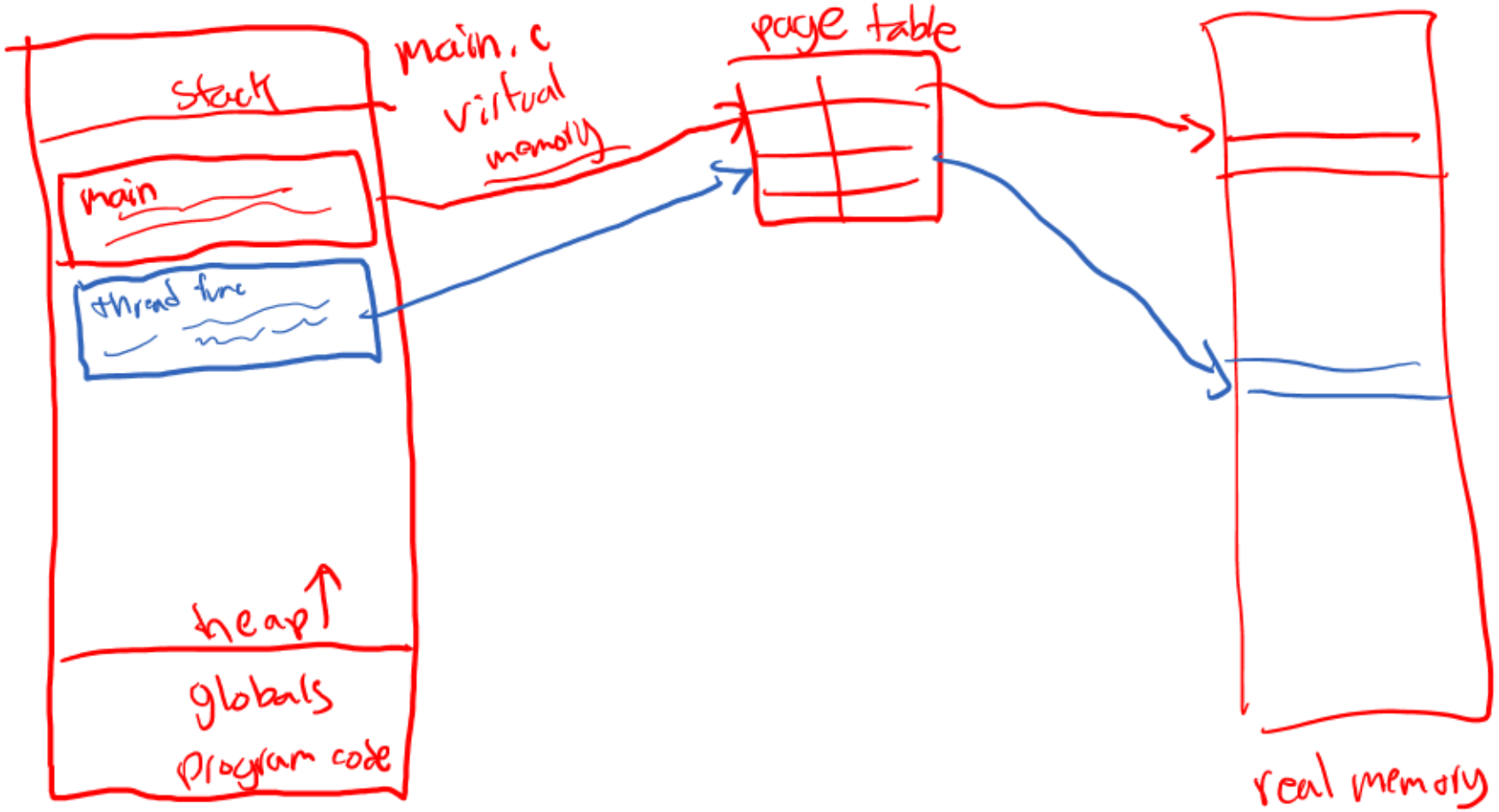
    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```



Virtual Memory Space of a Process with Threads



Context Switching

- if switching from 1 thread to another in the same process
  - PC
  - register values

takes longer

if switch from 1 thread to another in a diff process
- PC
- register values
- page table

Why Threads?

- UI responsiveness
- waiting on things like files
  - server - handle many users at once
  - shared computation

What are the max number of threads that could be running concurrently if I run this program from 5 terminals at the same time?

clicker.cs.illinois.edu

Q3

~Code~ 340

main.c

```c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

spins up 1 other thread

10

each can be running 2 threads

How many different page tables would the computer need to store to run this program from 5 terminals concurrently?

clicker.cs.illinois.edu

Q4

~Code~ 340

main.c

```c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

spins up another thread

10 threads going at max

5 one per process

With a 1 core CPU how many processes could I run in parallel?

same time

clicker.cs.illinois.edu

Q5

~Code~

340



main.c

```c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

With a 1 core CPU how many threads could I run in parallel?

clicker.cs.illinois.edu

Q6

~Code~ 340



1 PC

1 set of registers

1

main.c

```c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

Threading Example 0b10

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void *compute(void* arg){
    int* ptr = (int*)arg;
    printf("my value is %i\n", *ptr);
    return NULL;
}

int main() {
    pthread_t threads[2];
    int args[2] = {0, 1};

    for(int i = 0; i < 2; i++){
        pthread_create(&threads[i], NULL, compute, &args[i]);
    }
    for(int i = 0; i< 2; i++){
        pthread_join(threads[i], NULL);
    }
}
```

2 thread objects

0, 1

What will this code print?

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void *compute(void* arg){
    int* ptr = (int*)arg;
    sleep(*ptr);
    printf("%i ", *ptr);
    return NULL;
}

int main() {
    pthread_t threads[6];
    int args[6] = {5, 10, 1, 4, 2, 7};

    for(int i = 0; i < 6; i++){
        pthread_create(&threads[i], NULL, compute, &args[i]);
    }

    for(int i = 0; i < 6; i++){
        pthread_join(threads[i], NULL);
    }

}
```

clicker.cs.illinois.edu

Q7

~Code~
340

Without pthread_join what would print?

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <unistd.h>
4
5  void *compute(void* arg){
6      int* ptr = (int*)arg;
7      sleep(*ptr);
8      printf("%i ", *ptr);
9      return NULL;
10 }
11
12 int main() {
13     pthread_t threads[6];
14     int args[6] = {5, 10, 1, 4, 2, 7};
15
16     for(int i = 0; i < 6; i++){
17         pthread_create(&threads[i], NULL, compute, &args[i]);
18     }
19 }
```

sleep for 1 second

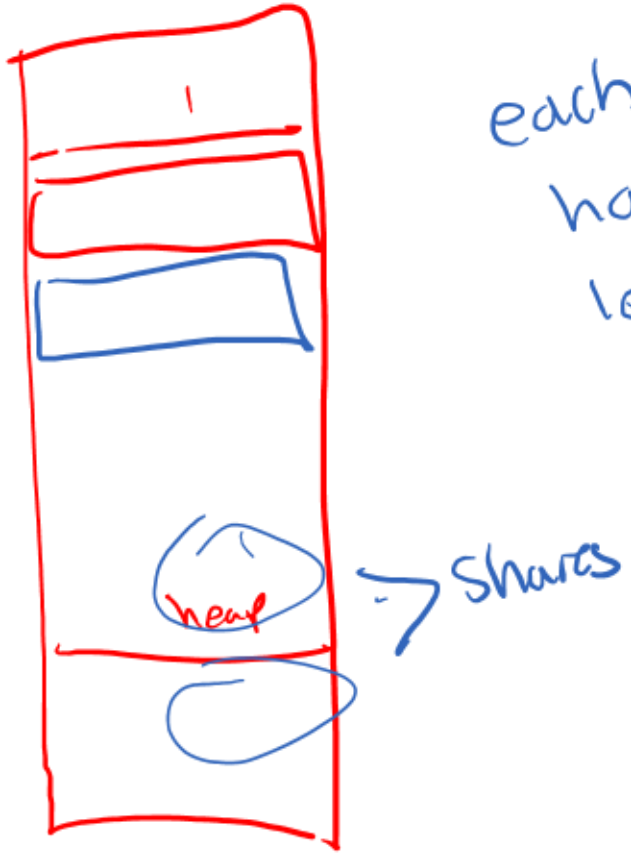
clicker.cs.illinois.edu

Q8

~Code~ 340

nothing

Threads within a Process



each thread has its own local variables

shares

Threading Example 0b11

```
5  int arr[2] = {0, 0};
6  int counter = 1;
7  void* compute(void* arg){
8      for(int i = 0; i < 2; i++){
9          if(arr[i] == 0){
10             arr[i] = counter;
11             counter++;
12         }
13     }
14     return NULL;
15 }
16
17 int main() {
18     pthread_t threads[2];
19     for(int i = 0; i < 2; i++){
20         if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
21             return 1;
22         }
23     }
24     for(int i = 0; i < 2; i++){
25         pthread_join(threads[i],NULL);
26     }
27     printf("%i, %i", arr[0], arr[1]);
28 }
```

print 1,2
2,3









Race Condition

Things are racing to access shared resources causing an non-deterministic result.

- multiple threads
- shared resource

- non deterministic result

sometimes a bug
sometimes it is fine

Is there a Race Condition?

```
5  void* compute(void* arg){
6      int arr[2] = {0, 0};
7      int counter = 1;
8      for(int i = 0; i < 2; i++){
9          if(arr[i] == 0){
10             arr[i] = counter;
11             counter++;
12         }
13     }
14     return NULL;
15 }
16
17 int main() {
18     pthread_t threads[2];
19     for(int i = 0; i < 2; i++){
20         if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
21             return 1;
22         }
23     }
24     for(int i = 0; i < 2; i++){
25         pthread_join(threads[i],NULL);
26     }
27     printf("%i, %i", arr[0], arr[1]);
28 }
```

✓ - multiple threads

✗ - shared resources

✗ - undeterministic results

clicker.cs.illinois.edu

Q9

~Code~ 340

Is there a Race Condition?

```
5  int arr[2] = {0, 3};
6  void* compute(void* arg){
7      int res = 5;
8      for(int i = 0; i < 2; i++){
9          if(arr[i] == 0){
10             res = res + 3;
11         }
12     }
13     return NULL;
14 }
15
16 int main() {
17     pthread_t threads[2];
18     for(int i = 0; i < 2; i++){
19         if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
20             return 1;
21         }
22     }
23     for(int i = 0; i < 2; i++){
24         pthread_join(threads[i],NULL);
25     }
26 }
```

√ threads?
√ shared ?
X undeterministic?

printed arr

clicker.cs.illinois.edu

Q10

~Code~
340

no

Is there a Race Condition?

```
5  int arr[2] = {0, 0};
6  void* compute(void* arg){
7      int res = 5;
8      for(int i = 0; i < 2; i++){
9          if(arr[i] == 0){
10             arr[i]++;
11         }
12     }
13     return NULL;
14 }
15
16 int main() {
17     compute(NULL);
18     compute(NULL);
19     compute(NULL);
20     compute(NULL);
21     printf("%i, %i", arr[0], arr[1]);
22 }
```

✗ threads
✓ shared
✗ undeterministic

clicker.cs.illinois.edu

Q11

~Code~
340

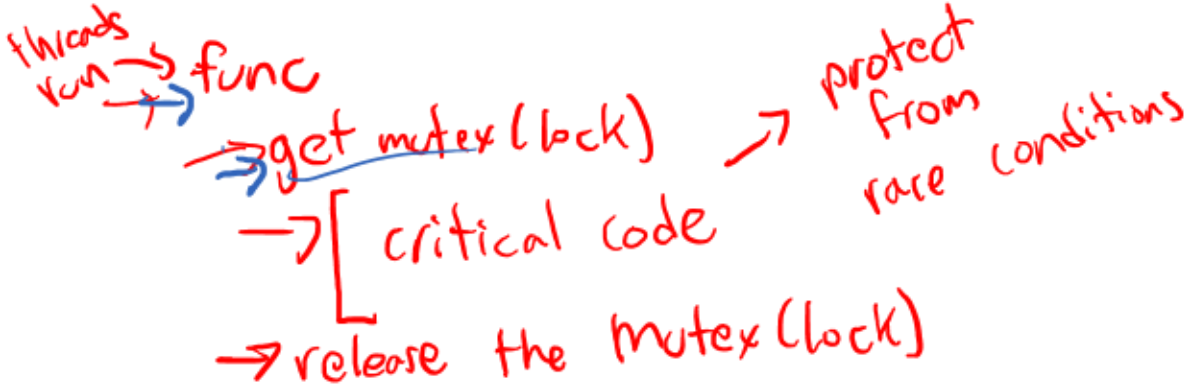
no

Preventing Race Conditions - Synchronization

- make code predictable and correct

Mutex - mutual exclusion lock

Make global mutex object



Static Mutex Example

```
5  pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
6  int arr[2] = {0, 0};
7  int counter = 1;
8  void* compute(void* arg){
9      pthread_mutex_lock(&mut);
10     for(int i = 0; i < 2; i++){
11         if(arr[i] == 0){
12             arr[i] = counter;
13             counter++;
14         }
15     }
16     pthread_mutex_unlock(&mut);
17     return NULL;
18 }
19
20 int main() {
21     pthread_t threads[2];
22     for(int i = 0; i < 2; i++){
23         if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
24             return 1;
25         }
26     }
27     for(int i = 0; i < 2; i++){
28         pthread_join(threads[i],NULL);
29     }
30     printf("%i, %i", arr[0], arr[1]);
31 }
```

What do mutexes give us? What do we lose?

- give us consistency
- correctness

lose
- speed

What would happen if I added a mutex?

```
5  void *sort(void* arg) {
6      int *ptr = (int*)arg;
7      sleep(*ptr);
8      printf("%i ", *ptr);
9      return NULL;
10 }
11
12 int main() {
13     int arr[6] = {5, 10, 1, 4, 2, 7};
14     pthread_t threads[6];
15
16     for (int i = 0; i < 6; i++) {
17         if (pthread_create(&threads[i], NULL, sort, &(arr[i])) != 0) {
18             perror("Failed to create thread");
19             return 1;
20         }
21     }
22
23     //wait for both threads to finish
24     for (int i = 0; i < 6; i++) {
25         pthread_join(threads[i], NULL);
26     }
27 }
```

Big Picture Perspective

Limitations of Reality

System

Processes and Threads

Process

Thread

Synchronization

Race Condition

Mutex